

CC1002 Introducción a la Programación - Sección 1

Tarea 1

Prof. Benjamin Bustos

Fecha de entrega: lunes 7 de septiembre de 2026 a las 23:59

Es obligatorio el uso de la receta de diseño visto en cátedra para implementar todas las funciones.

La tarea debe realizarse en forma individual. No olvide leer el documento de Política de Colaboración publicado en U-Cursos.

Operación de suavizado en secuencias de números

Suponga que dispone de una secuencia de números que provienen de alguna fuente, por ejemplo valores de una acción bursátil, mediciones climatológicas, intensidad de los píxeles de una imagen, etc. En ciertos contextos, antes de proceder a analizar los datos es conveniente realizar una etapa de pre-procesamiento, en donde uno se asegura que los datos cumplan con un cierto estándar, se normalicen sus valores, se imputen datos faltantes, se simplifiquen, o se les realice alguna otra operación necesaria.

Una de estas operaciones de pre-procesamiento se denomina *suavizado*. Esta operación permite reducir ruido en los datos y disminuir discontinuidades abruptas. Los valores ruidosos pueden deberse a diversas causas, como por ejemplo: falla en los instrumentos de medición; problemas en el ingreso de datos; problemas en la transmisión de datos; limitaciones de la tecnología; inconsistencias en las convenciones de etiquetado. El suavizado permite aliviar el ruido y mejorar la eficacia de los algoritmos que posteriormente procesen los datos.

Una forma simple de suavizado consiste en ordenar los datos y particionarlos en segmentos del mismo ancho. Luego, cada segmento se suaviza con alguna regla predefinida. Por ejemplo, suponga que se dispone de la siguiente secuencia de valores enteros:

14, 18, 19, 25, 31, 31, 34, 35, 36, 38, 39.

Se decide que $n = 4$ será el tamaño de los segmentos. Por lo tanto, la secuencia queda dividida en los tres siguientes segmentos:

14, 18, 19, 25.

31, 31, 34, 35.

36, 38, 39.

Note que el último segmento queda de un largo distinto $\leq n$, dado que el largo de la secuencia no es divisible en forma exacta por n . Luego, el proceso de suavizado consiste en reemplazar algunos o todos los valores de cada segmento por algún valor o valores representativos. Por ejemplo, una opción es reemplazar los valores de cada segmento por la parte entera de la media de los valores del segmento. Para nuestro ejemplo, el suavizado resultante sería:

19, 19, 19, 19.

32, 32, 32, 32.

37, 37, 37.

Note que, en este ejemplo, para el último segmento la media se calcula sólo sobre sus tres valores, no sobre cuatro valores como para los segmentos previos.

Representación de una secuencia y sus segmentos

Para representar la información de una secuencia de valores, se tomarán los siguientes supuestos:

- La secuencia de números debe ser una secuencia de enteros positivos no decreciente.
- Las secuencias contendrán números enteros de dos dígitos (valores en el rango $[10, 99]$).
- Una secuencia se representará por un número entero. Cada dos dígitos de este número se representará un valor de la secuencia. Los primeros dos dígitos menos significativos corresponden al primer número de la secuencia, los siguientes dos dígitos menos significativos corresponden al segundo número de la secuencia, etc. Esto implica que todo número que representa a una secuencia debe tener un número par de dígitos.

- El tamaño del segmento debe ser un número entero $n \geq 1$.
- Un segmento también se representa por un número entero de hasta $2n$ dígitos, siguiendo las mismas reglas que para una secuencia.

Siguiendo estas reglas, la secuencia del ejemplo de la parte anterior se representa con el número:

3938363534313125191814

Si $n = 4$, sus segmentos corresponden a los números:

25191814

35343131

393836

En el resto del enunciado, cuando se hable de una “secuencia” o “segmento” se estará refiriendo a su número entero asociado. Una secuencia o segmento sin valores se representa con el número 0.

Tarea

Implemente un módulo de Python que provea las siguientes funciones:

1. *primerSegmento(secuencia, n)*. Función que recibe una secuencia y tamaño del segmento válidos, y devuelve el número correspondiente al primer segmento de la secuencia.
2. *restoSegmentos(secuencia, n)*. Función que recibe una secuencia y tamaño del segmento válidos, y devuelve el número correspondiente a la secuencia donde se le quitó el primer segmento.
3. *contar(secuencia)*. Función que recibe una secuencia válida y devuelve la cantidad de números en la secuencia.
4. *estaOrdenada(secuencia)*. Función que recibe una secuencia válida y devuelve un valor booleano, indicando si la secuencia está o no ordenada (enteros no decrecientes).
5. *proceso(secuencia, n)*. Función que recibe una secuencia y el tamaño del segmento. Si alguno de estos valores es inválido, la función devuelve -1. Si no, la función devuelve un número que representa a la secuencia con sus valores suavizados.

6. *textoSegmento(segmento)*. Función que recibe un segmento, y devuelve un texto (String) con los valores del segmento, ordenados, en una misma línea, separados por coma.

Ud. debe implementar en forma obligatoria en su módulo todas estas funciones solicitadas. Además, Ud. debe implementar todas las funciones auxiliares que requiera, en caso por ejemplo que para implementar algunas de las funciones solicitadas necesite descomponer el problema en subproblemas más pequeños. Recuerde que cada función implementada debe resolver un único problema o realizar una única tarea, si está intentando resolver más de un problema a la vez piense en cómo se podría descomponer en subproblemas (esto será evaluado).

Finalmente, implemente en otro módulo un programa interactivo que pregunte por el número que representa una secuencia y el tamaño de un segmento, que verifique que los datos ingresados son correctos, y que luego muestre los valores de los segmentos de la secuencia original y de la secuencia suavizada. Para implementar el programa interactivo, use el módulo con las funciones programadas, agregando las funciones o procedimientos auxiliares que pudiese requerir.

Un ejemplo de ejecución del programa interactivo:

```
Secuencia? 3938363534313125191814
n? 4
Valores originales:
14, 18, 19, 25
31, 31, 34, 35
36, 38, 39
Valores despues de proceso de suavizado:
19, 19, 19, 19
32, 32, 32, 32
37, 37, 37
```

Otro ejemplo:

```
Secuencia? 3938363534313125191814
n? 5
Valores originales:
14, 18, 19, 25, 31
31, 34, 35, 36, 38
39
```

Valores despues de proceso de suavizado:

21, 21, 21, 21, 21

34, 34, 34, 34, 34

39

Un último ejemplo:

Secuencia? 393836353431312519181

n? -2

Datos ingresados no validos

Para el programa interactivo, puede suponer que el usuario siempre entrega como input números enteros.

Testing

Debe implementar al menos cuatro tests para toda función programada en el módulo solicitado, incluyendo cualquier función auxiliar que haya implementado. No es necesario incluir testing para el programa interactivo ni para los procedimientos que requiera implementar.

Asignación de puntajes

- Receta de diseño: 1.5 puntos (repartido entre todas las funciones solicitadas).
 - Contrato: 0.4 ptos. en total
 - Descripcion: 0.15 ptos. en total
 - Ejemplo: 0.15 ptos. en total
 - Testing: 0.8 ptos. (cuatro tests en cada una de las funciones programadas)
- Funciones solicitadas para módulo: 3.5 ptos en total.
 - Función primerSegmento: 0.3 pto.
 - Función restoSegmentos: 0.3 pto.
 - Función contar: 0.4 ptos.
 - Función estaOrdenada: 0.7 ptos.
 - Función proceso: 1.0 ptos.

- Función `textoSegmento`: 0.8 ptos.
- Módulo programa interactivo: 1. pto.
- Descuentos globales:
 - Uso de buenas metodologías de programación: hasta -0.5 ptos.
 - Descomposición funcional: hasta -0.5 ptos.

Planificación para realizar la tarea

Al momento de la publicación de este enunciado, Ud. no dispone aún de todas las herramientas de programación para resolver la tarea completa, pero si las tendrá para la semana previa a la entrega. Se le recomienda la siguiente planificación para ir avanzando parcialmente en la tarea cada semana:

- Leer y entender bien el enunciado: Semana 2.
- Diseñar una descomposición funcional adecuada para todas las funciones solicitadas: Semanas 2 y 3.
- Funciones *`primerSegmento(secuencia, n)`* y *`restoSegmentos(secuencia, n)`*: Semana 2.
- Funciones restantes y programa interactivo: Semana 3 en adelante.

Notas importantes

- Para programar su tarea, sólo puede utilizar las herramientas de Python que hayamos visto en el curso. En particular, no se pueden usar instrucciones de Python que no se hayan visto en clase de cátedra, no estén en el apunte entre los Capítulos 1 a 7, o no hayan sido especificadas en este enunciado. Soluciones que no cumplan con este punto, serán evaluadas con la nota mínima.
- Se evaluará el uso de buenas metodologías de programación (receta de diseño, nombres de funciones y variables representativos, evitar duplicar código, etc.).
- Antes de ponerse a programar, piense en cuál sería una descomposición funcional adecuada para cada función solicitada.

- Esta prohibido utilizar ambientes de desarrollo con IA integrada, por ejemplo Colab. Utilice IDLE que se instala junto con Python para programar su tarea.
- Implemente el módulo solicitado en el archivo `moduloTarea1.py`, y el programa interactivo en el archivo `programaTarea1.py`.
- Su tarea debe respetar *estrictamente* los nombres de funciones, parámetros, tipos de datos y reglas descritos en el enunciado.
- No olvide leer y seguir las reglas descritas en el documento de Política de Colaboración publicado en U-Cursos.
- Dudas sobre el enunciado pueden postearlas en el foro del curso. *ESTÁ PROHIBIDO COPIAR CÓDIGO FUENTE DE SU TAREA EN EL FORO*. Si Ud. copia código de su tarea en el foro, aunque sea para preguntar por un error, tendrá nota 1.0.
- La tarea debe entregarse a través de la página de U-Cursos, sección Tareas. Entregas durante el período de entrega con atraso no tienen descuento. *NO SE ACEPTARÁN TAREAS FUERA DE PLAZO NI ENTREGADAS POR OTROS MEDIOS*.